

Rapport individuel - Agentic RAG avec LangGraph

1. Demarche suivie

Le projet realise un systeme Agentic RAG dans le domaine de l'education informatique. La base documentaire est composee de documents Markdown sur les systemes RAG, LangGraph, l'evaluation, la securite et l'architecture du projet. Les documents sont nettoyes, decoupees en fragments puis vectorises avec TF-IDF. Ce choix permet une execution locale simple et reproductible.

2. Fonctionnement du systeme

L'utilisateur pose une question via la CLI. Le graphe LangGraph initialise le state, lance la recherche documentaire, evalue la pertinence des fragments recuperes, reformule la requete si le score est faible, puis genere une reponse avec les sources. Le state contient la question, la requete, les documents, la note de pertinence, la reponse, les sources, le temps d'execution, le nombre d'iterations et la memoire.

Les outils developpes sont `retrieve_documents`, `grade_retrieval`, `build_context` et `compact_sources`. Le graphe contient les noeuds `plan`, `retrieve`, `grade`, `rewrite`, `generate` et `finalize`. Cette architecture respecte l'approche Agentic RAG car elle introduit une decision conditionnelle et une boucle de correction au lieu d'une simple chaine lineaire.

3. Resultats d'evaluation

L'evaluation utilise 20 questions : 10 simples et 10 complexes. Les questions simples testent la recuperation directe d'information. Les questions complexes demandent une synthese entre plusieurs documents, par exemple comparer un RAG lineaire et un Agentic RAG ou analyser les limites du systeme.

Le script `scripts/run_evaluation.py` mesure le temps de reponse, le nombre d'iterations et le score moyen de similarite des documents recuperes. Les resultats details sont generes dans `outputs/evaluation_results.json` et synthetises dans `outputs/evaluation_report.md`.

4. Limites et pistes d'amelioration

La vectorisation TF-IDF est simple et interpretable, mais elle gere moins bien les synonymes que des embeddings denses. En absence de cle API, le generateur local produit une reponse extractive ; avec une cle OpenAI, le projet peut utiliser un vrai LLM via `langchain-openai`.

Les ameliorations possibles sont l'ajout d'un reranker, l'utilisation d'embeddings semantiques, une interface web, une evaluation humaine de la fidelite et une protection plus avancee contre les injections de prompt dans les documents.

Synthese chiffrée

Evaluation executee sur 20 questions. Temps moyen: 55.32 ms. Score moyen de recuperation: 0.1076.